

BranchPad - Development Plan

Requirements

Overview

A lightweight desktop app for developers to track GitHub branches and their associated PRs. Similar to Notepad++ in spirit - fast, minimal, stays out of the way.

Core Features

1. **Repository Management** - Add repos manually via GitHub URL (e.g., `https://github.com/owner/repo`) - View list of tracked repos - Remove repos when no longer needed
 2. **Branch Tracking** - Add branch names manually (paste multiple branches at once) - Auto-discover PR for each branch using `gh pr list --head <branch>` - Display branch with its associated PR
 3. **PR Status Indicators** -  Draft -  Ready for review -  Approved (show count) -  Pending comments (show count) -  Merged (auto-move to history)
 4. **Copy to Clipboard** - Copy branch name - Copy PR number - Copy PR URL
 5. **Saved PRs Section** - Bookmark any PR (not just your branches) - Manual add by PR URL or number - Same status indicators
 6. **Sticky Notes** - Simple text area per branch/PR - Quick notes, reminders, context - Autosave on every keystroke
 7. **History Section** - All merged PRs moved here automatically - Searchable by:
 - Branch name
 - PR title
 - Repository
 - Date range
 - Clear history option
 8. **Auto-refresh** - Auto-fetch PR status on app open - Background refresh (configurable interval) - Manual refresh button
-

Technology Stack

Component	Technology	Reason
Desktop Framework	Electron	Mature ecosystem, cross-platform
UI Framework	React 18	Component-based, fast
Build Tool	Vite	Fast HMR, modern bundler
Database	SQLite (better-sqlite3)	Local persistence, autosave
State Management	Zustand	Lightweight, simple API
Styling	Tailwind CSS	Fast development, consistent
GitHub API	GitHub CLI (gh)	Uses existing auth, no OAuth needed

Architecture

```
BranchPad/
├─ electron/           # Main Electron process
|   ├─ main.js         # App lifecycle, windows, tray
|   ├─ preload.js      # IPC bridge (contextBridge)
|   └─ database.js     # SQLite setup & queries
├─ src/                # React renderer (Vite)
|   ├─ components/     # UI components
|   |   ├─ layout/     # Sidebar, Header, etc.
|   |   ├─ repos/      # Repository management
|   |   ├─ branches/   # Branch list & management
|   |   ├─ prs/        # PR display & status
|   |   ├─ notes/      # Sticky notes editor
|   |   └─ history/    # History view with search
|   ├─ hooks/          # Custom hooks
|   |   ├─ useRepos.js # Repo CRUD operations
|   |   ├─ useBranches.js # Branch CRUD operations
|   |   ├─ usePRs.js   # PR fetching via gh CLI
|   |   └─ useNotes.js # Notes autosave
|   ├─ stores/         # Zustand stores
|   |   ├─ repoStore.js
|   |   ├─ branchStore.js
|   |   └─ settingsStore.js
|   ├─ utils/          # Helpers
|   |   ├─ gh.js       # GitHub CLI wrappers
|   |   └─ clipboard.js # Copy utilities
|   ├─ App.jsx
|   ├─ main.jsx
|   └─ index.css       # Tailwind imports
├─ package.json
├─ vite.config.js
├─ tailwind.config.js
└─ README.md
```

Database Schema (SQLite)

```
-- Repositories tracked
CREATE TABLE repositories (
  id INTEGER PRIMARY KEY,
  owner TEXT NOT NULL,
  name TEXT NOT NULL,
  url TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(owner, name)
);

-- Branches tracked (manually added by user)
CREATE TABLE branches (
  id INTEGER PRIMARY KEY,
  repo_id INTEGER NOT NULL,
  name TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (repo_id) REFERENCES repositories(id) ON DELETE CASCADE,
  UNIQUE(repo_id, name)
);

-- PRs associated with branches
CREATE TABLE pull_requests (
  id INTEGER PRIMARY KEY,
  github_id INTEGER UNIQUE,
  branch_id INTEGER,
  number INTEGER,
  title TEXT,
  state TEXT, -- open, closed, merged
  is_draft BOOLEAN,
  approvals_count INTEGER DEFAULT 0,
  pending_comments_count INTEGER DEFAULT 0,
  html_url TEXT,
  merged_at DATETIME,
  last_fetched DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE
);

-- Saved/Bookmarked PRs (not tied to branches)
CREATE TABLE saved_prs (
```

```
id INTEGER PRIMARY KEY,
github_id INTEGER UNIQUE,
repo_owner TEXT,
repo_name TEXT,
number INTEGER,
title TEXT,
state TEXT,
html_url TEXT,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Sticky notes
CREATE TABLE sticky_notes (
  id INTEGER PRIMARY KEY,
  target_type TEXT, -- 'branch', 'pr', 'saved_pr'
  target_id INTEGER,
  content TEXT,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- App settings
CREATE TABLE settings (
  key TEXT PRIMARY KEY,
  value TEXT
);

-- History (merged PRs)
CREATE TABLE pr_history (
  id INTEGER PRIMARY KEY,
  github_id INTEGER UNIQUE,
  repo_owner TEXT,
  repo_name TEXT,
  branch_name TEXT,
  number INTEGER,
  title TEXT,
  html_url TEXT,
  merged_at DATETIME,
  moved_to_history_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Implementation Phases

Phase 1: Project Setup

- Initialize Electron + React + Vite project
- Set up SQLite database with schema
- Basic window configuration
- Tailwind CSS configuration
- Project structure scaffolding

Deliverable: App launches with blank window

Phase 2: Core UI Shell

- Main window layout with sidebar navigation
- Views: Repositories, Branches, Saved PRs, History, Settings
- Responsive layout
- Theme support (light/dark)

Deliverable: App shows navigation, can switch between empty views

Phase 3: Repository Management

- Add repo form: paste GitHub URL, parse owner/repo
- Display list of tracked repos
- Store repos in SQLite
- Remove repo functionality (cascades to branches)

Deliverable: User can add/remove repos

Phase 4: Branch Management

- Add branches manually (textarea for multiple branches, one per line)
- Store branches linked to repos
- Display branch list grouped by repo
- Remove branch functionality

Deliverable: User can add/remove branches per repo

Phase 5: GitHub CLI Integration

- Check `gh` CLI availability on startup
- Use `gh pr list --repo owner/repo --head <branch>` to find PR
- Parse `gh pr view <pr-number> --json` for:
 - State (open/closed/merged)
 - Draft status
 - Review decision (APPROVED/CHANGES_REQUESTED/REVIEW_REQUIRED)
 - Review comments count
- Auto-discover PR when branch is added
- Display PR status with visual indicators

Deliverable: PRs auto-discovered and displayed with status

Phase 6: PR Features

- Status badges:  Draft,  Ready,  Approved,  Comments,  Merged
- Copy buttons for branch name, PR number, PR URL
- Saved PRs section (add by PR number or URL)
- Auto-refresh PR data on interval
- Manual refresh button

Deliverable: Full PR tracking with copy functionality

Phase 7: Sticky Notes

- Note editor component per branch/PR
- Autosave on keystroke with debounce (500ms)
- Display note indicator on items with notes

Deliverable: Notes persist and survive app restart

Phase 8: History & Search

- Auto-move merged PRs to history table
- History view with search bar

- Filter by: branch name, PR title, repo, date range
- Clear history option

Deliverable: Merged PRs archived, searchable history

Phase 9: Polish & Packaging

- System tray icon (minimize to tray)
- App icon & branding
- Settings panel:
 - Refresh interval
 - Theme toggle
 - Clear data option

Deliverable: Portable Windows executable — no installation, no admin rights, data stored beside the `.exe`

Key Dependencies

```
{
  "name": "branchpad",
  "version": "1.0.0",
  "main": "electron/main.js",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "electron:dev": "concurrently \"vite\" \"wait-on http://localhost:5173 && electron .\"",
    "electron:build": "vite build && electron-builder",
    "start": "electron ."
  },
  "dependencies": {
    "electron": "^28.0.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "better-sqlite3": "^9.2.0",
    "zustand": "^4.4.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.0",
    "vite": "^5.0.0",
    "electron-builder": "^24.9.0",
    "tailwindcss": "^3.4.0",
    "postcss": "^8.4.0",
    "autoprefixer": "^10.4.0",
    "concurrently": "^8.2.0",
    "wait-on": "^7.2.0"
  }
}
```

GitHub CLI Commands Reference

```
# Check if gh is installed and authenticated
gh auth status

# List PRs for a branch
gh pr list --repo owner/repo --head branch-name --state all --json number,title,state,isDraft,url

# Get detailed PR info
gh pr view 123 --repo owner/repo --json number,title,state,isDraft,reviewDecision,comments,mergedAt,u

# Get PR reviews
gh api repos/owner/repo/pulls/123/reviews --jq '.[0] | {user: .user.login, state: .state}'

# Get PR review comments count
gh pr view 123 --repo owner/repo --json comments --jq '.comments'
```

Verification Checklist

Phase	Test
1	App launches and shows window
2	Navigation between views works
3	Can add/remove repos via URL
4	Can add/remove branches per repo
5	PRs auto-discovered for branches
6	Copy to clipboard works, refresh works
7	Notes save automatically
8	Merged PRs in history, search works
9	App installs and runs standalone

Portable Packaging

Why portable?

The user's office laptop blocks installing new `.exe` files. A portable build runs directly without an installer or admin rights.

Approach

Use `electron-builder` with the `portable` Windows target. It produces a single self-extracting `.exe` (like `7z-sfx`):

Property	Value
Output	<code>dist/BranchPad-Portable-1.0.0.exe</code>
Install required	✗ None
Admin rights	✗ None
First run	Extracts itself to <code>%TEMP%</code> (default) or kept-in-place
Data location	<code>Portable/</code> folder created next to the .exe (moves with the app)

electron-builder config

```
{
  "build": {
    "appId": "com.branchpad.app",
    "productName": "BranchPad",
    "directories": { "output": "dist" },
    "files": ["dist/**/*", "electron/**/*", "package.json"],
    "win": {
      "target": [{ "target": "portable", "arch": ["x64"] }],
      "artifactName": "BranchPad-Portable-${version}.exe"
    },
  },
  "portable": { "artifactName": "BranchPad-Portable-${version}.exe" }
}
```

Key code change — data beside the exe (true portability)

In the main process, detect portable mode and store the SQLite DB inside the `Portable/` folder next to the executable so data travels with the app:

```
// electron/main.js
const { app } = require('electron');
const path = require('path');
const fs = require('fs');

function getDataDir() {
  // Portable builds set this env var to the folder containing the exe
  const portableDir = process.env.PORTABLE_EXECUTABLE_DIR || process.env.PORTABLE_EXECUTABLE_FILE;
  if (portableDir) {
    const dataDir = path.join(portableDir, 'Portable', 'Data');
    fs.mkdirSync(dataDir, { recursive: true });
    return dataDir;
  }
  // Normal (dev) mode: standard user-data dir
  return app.getPath('userData');
}
```

Every DB file, settings, and cached PR data goes through `getDataDir()`.

Limitations (expected & accepted)

- ⚠️ **No auto-update** — portable builds don't support `electron-updater`. User re-downloads a new `.exe` for updates.
- ⚠️ **One instance at a time** — running two copies on the same folder risks SQLite lock conflicts; enable `app.requestSingleInstanceLock()`.
- ⚠️ Some corporate policies (AppLocker/allowlists) block running **any** unapproved `.exe`, portable or not. If that's the case, nothing exe-based will run. Fallback: app could also be run as a plain local web app (Vite dev/build + local server, same React code, data in a local JSON/SQLite via the browser) — but that's a separate packaging path, only if truly blocked.

Scripts

```
"electron:build": "vite build && electron-builder --win --portable"
```

Verification

1. Run `npm run electron:build`
 2. Copy `dist/BranchPad-Portable-1.0.0.exe` to a fresh folder
 3. Double-click — app opens **without any installer**
 4. Add a repo + branch; force-close the app
 5. Confirm a `Portable/Data/` folder appears next to the `.exe` and the DB survives restart
 6. Move the whole folder to another location — data still there
-

Prerequisites

Before running BranchPad, ensure you have: 1. **Node.js** v18+ installed (build machine) 2. **GitHub CLI (gh)** installed on the machine that runs the app: <https://cli.github.com/> 3. **GitHub CLI authenticated**: Run `gh auth login`

Note for the office laptop: BranchPad runs portably with no install, and `gh` CLI is already installed & authenticated on the office laptop — so no extra setup needed there. The app shells out to `gh` for all PR data.

Next Steps

1. Initialize npm project in `BranchPad/` folder
2. Install dependencies
3. Set up Electron main process
4. Create SQLite database module
5. Build React UI with Vite
6. Implement repo and branch management
7. Integrate GitHub CLI for PR discovery